

LCA Product Retrieval System

Design Document

Author: Jay Parmar

Role: Senior AI Engineer

Date: June 2026

Version: 1.0 — Final Submission

Table of Contents

- 1. Executive Summary
- 2. Problem Statement & Dataset Analysis
- 3. System Architecture
- 4. Core Design Decisions & Justifications
- 5. Non-Functional Requirements
 - 5.1 Security
 - 5.2 Reliability & Fault Tolerance
 - 5.3 Scalability
 - 5.4 Latency
 - 5.5 Accuracy & Quality
- 6. Model Evaluation Strategy
- 7. Retrieval Improvement Roadmap
- 8. Proposed Production SaaS Architecture
- 9. Technology Decisions Summary
- Appendix A** Notebook Execution Screenshots

1. Executive Summary

This document presents the design and implementation of a hybrid semantic retrieval system built to power customer-facing AI agents operating over a catalogue of 200 construction materials products. The system delivered here solves this through intelligent query routing over a hybrid BM25 + dense semantic pipeline with post-retrieval constraint verification.

Headline Results

Metric	BM25 Only	Dense Only	Hybrid (Auto)
Hit@1	0.829	0.438	0.924
Hit@3	0.943	0.543	1.000
Hit@5	0.952	0.610	1.000
MRR	0.881	0.497	0.959

2. Problem Statement & Dataset Analysis

Customer-facing AI procurement agents require fast, accurate product lookup over a catalogue where all product information is stored as a single unstructured text field with no consistent schema. The absence of structured fields is the defining constraint: before any retrieval can happen, the system must reconstruct structure from noise.

2.1 The Dataset

- 200 products across 10 perfectly balanced categories (20 per category): Cement, Steel Cables, Insulation, Concrete Blocks, Paint, Plasterboard, Timber, Rebar, Waterproofing, Pipes
- Single field: raw_text — all product information in one unstructured string
- 5 distinct serialisation formats, none labelled in the data:

Format Name	Example Pattern
comma	"Heidelberg Materials Portland Max, C30/37, 25kg, declaration EPD202400001, sku CEM-1001..."
pipe	"Premium Mix Plus manufacturer Heidelberg Materials EPD202400002 C25/30..."
slash	"Rapid Set Eco by Holcim / cement / CEM I 42.5N / 25kg / CEM-1003..."
epd_first	"EPD202400004 Buzzi Unicem Rapid Set Standard C30/37 25kg... code CEM-1004"
product_colon	"Product: LiftLine Eco; Maker: Prysmian; Specs: 8mm; 1770 MPa; ID: EPD202400021"

2.2 Query Diversity — The Core Challenge

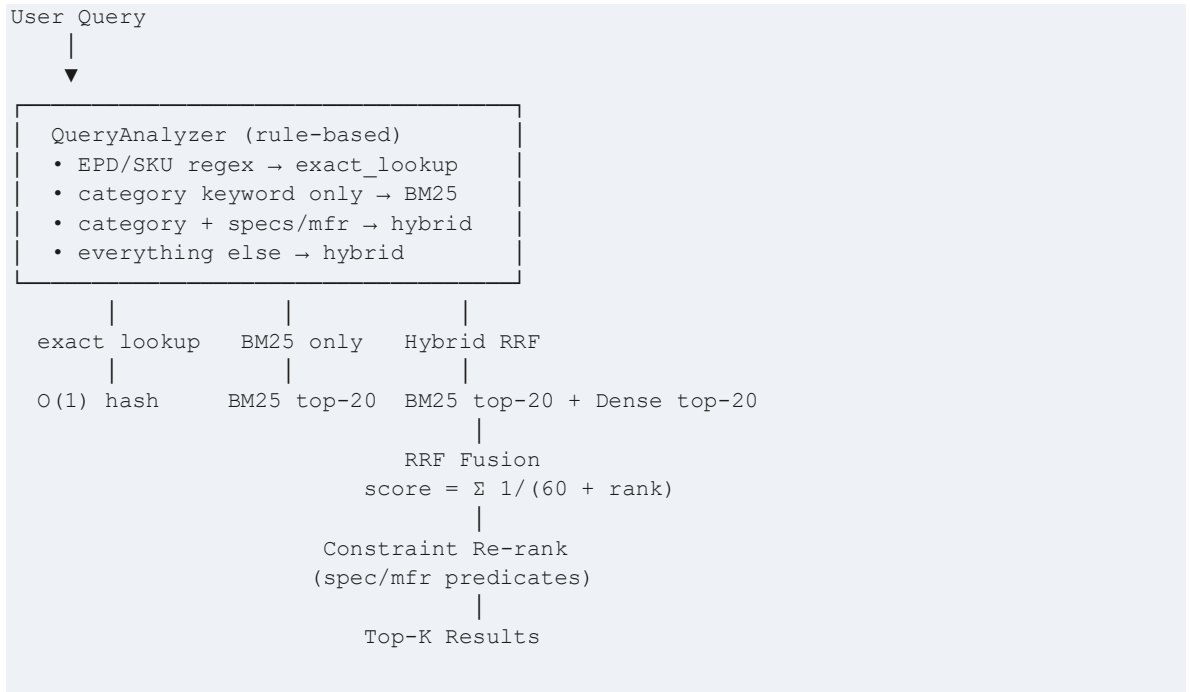
The fundamental insight driving every design decision is that user queries fall into qualitatively different retrieval problems. A system optimised for one type degrades on others:

Query Type	Example	Challenge
Vague category	cement	Any of 20 products should rank highly
Manufacturer-constrained	cement Heidelberg Materials	Named entity buried in unstructured text
Exact ID	EPD202400001	Deterministic lookup; probabilistic ranking fails
Technical spec	steel cables 12mm 1770 MPa	Exact numeric tokens; semantic distance irrelevant
Semantic	fire-resistant board for cladding	No token overlap; requires semantic understanding
Multi-attribute	rebar ArcelorMittal B500B 16mm	Conjunction of constraints — all must hold

3. System Architecture

The system is a query-type-aware hybrid retrieval pipeline. Rather than routing all queries through the same retrieval stack, a lightweight rule-based analyzer classifies each query and selects the minimum-cost retrieval strategy that maximizes relevance for that query type.

3.1 Pipeline Overview



3.2 Preprocessing Pipeline

All product text is processed through a 5-format-specific parser + universal regex fallback before indexing. The system reconstructs 12 structured fields from each raw text record:

Field	Coverage	Extraction Method
EPD ID	100%	Universal regex <code>EPD\d{9}</code>
SKU	100%	Universal regex <code>[A-Z]{2,6}-\d{3,5}</code>
Manufacturer	100%	3-pass bootstrap: explicit markers → known-name propagation → repeated leading prefix
Category	100%	Keyword taxonomy over normalized text
Dimensions (mm)	~60%	Numeric extractor with unit normalisation
Weight (kg)	~60%	Numeric extractor
Strength (MPa)	~40%	Numeric extractor
Grade	~50%	Pattern: <code>C30/37</code> , <code>B500B</code> , <code>PN16</code> , <code>CEM I 42.5N</code>
Certifications	~20%	Keyword set: <code>PEFC</code> , <code>FSC</code> , <code>A1</code> , <code>A2-s1,d0</code> , <code>CE</code>
Lambda (W/mK)	~20%	Thermal insulation only
Volume (L)	~30%	Numeric extractor
Color	~10%	Color keyword set (Paint category)

4. Core Design Decisions & Justifications

Every design choice below was made against an explicit rejected alternative. This section documents not only what was built, but why, so any future engineer inheriting this system can understand the trade-off space without re-deriving it.

4.1 Preprocessing Before Indexing

Parse raw_text into structured ProductRecord fields and embed the reconstructed normalised text, not the raw string.

4.2 BM25Okapi for Sparse Retrieval

BM25Okapi with $k_1=1.5$, $b=0.75$ over tokenised normalised text.

* In research paper Thakur et al. (2021) shows across 18 BEIR datasets that BM25 outperforms many dense-only models on **exact-match-heavy domains** such as product codes, manufacturer names, and technical specifications. For queries like "cement manufacturer Heidelberg" or "steel cables 12mm", BM25's term frequency weighting directly rewards documents containing those exact tokens. The Okapi variant with $k_1=1.5$, $b=0.75$ is the industry-standard parameterization validated across TREC benchmarks; k_1 controls term saturation and b controls document length normalization, both of which matter when product descriptions vary widely in length.

Link: Thakur et al. (2021) — "BEIR: A Heterogeneous Benchmark for Zero-Shot Evaluation of Information Retrieval Models" (arXiv:2104.08663).

4.3 BAAI/bge-small-en-v1.5 for Dense Retrieval

bge-small-en-v1.5 with L2-normalised embeddings stored as a 200×384 numpy float32 matrix.

Model	Params	Avg NDCG@10 (BEIR)	Trade-off
bge-small-en-v1.5 (chosen)	~33M params	~53	Asymmetric retrieval tuned, free, local

* Xiao et al. (SIGIR 2024) explicitly stated that the BGE small-scale models achieve "best performance among models of the same size" at release. IBM's Granite R2 paper independently confirms bge-small-en-v1.5 achieves BEIR NDCG@10 of **51.7**, beating e5-small-v2 (49.0) at identical parameter count — a +2.7-point margin in a fair size-matched comparison. Reimers & Gurevych (2019) establish that SBERT-style bi-encoders, which bge-small uses, reduce semantic similarity search from $O(n^2)$ cross-encoder inference to $O(n)$ dot-product lookup — critical for a system that must remain fast at query time. t 33.4M parameters and 384 dimensions, the model encodes 200 products in under one second on CPU, making the prototype fully reproducible without GPU infrastructure.

Link: Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2024). C-Pack: Packaged Resources to Advance General Chinese Embedding. SIGIR 2024. arXiv:2309.07597.

Link: Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019. arXiv:1908.10084.

4.4 Reciprocal Rank Fusion (RRF) for Hybrid Combination

Combine BM25 and dense ranked lists using RRF with $k=60$ (Cormack et al., 2009).

Formula: $RRF_score(d) = 1/(60 + rank_BM25(d)) + 1/(60 + rank_Dense(d))$

*Cormack et al. (2009) demonstrated that RRF consistently outperforms both individual rankers and more complex learned fusion methods, with **no training data required**. This is the decisive advantage for our system as it has no labelled query-product pairs, so any score-based learned fusion (e.g. linear interpolation, LambdaMART) cannot be calibrated.

Link:Cormack, G.V., Clarke, C.L.A., & Buettcher, S. (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. SIGIR 2009. pp. 758–759.

4.5 Intelligent Query Routing

Decision: Rule-based cascade classifies queries and routes to the minimum-cost strategy.

Query Pattern	Strategy	Reasoning
query matches EPD regex	exact_lookup	$O(1)$ hash scan; tokenisation can fragment IDs
query matches SKU regex	exact_lookup	$O(1)$ hash scan
bare category keyword	BM25-only	Semantic drift risk — 'cement' semantic neighbours include non-cement
category + spec/mfr	hybrid (RRF)	Both lexical precision and semantic coverage needed
general / ambiguous	hybrid (RRF)	Safe default; covers cases not matching above

4.6 Post-Retrieval Constraint Re-ranking

After initial retrieval, compile query-specified constraints (dimensions, strength, grade, certification, manufacturer, color) into verification predicates and promote products satisfying more constraints via stable sort over an enlarged candidate pool (≥ 40 candidates).

5. Non-Functional Requirements

A retrieval system for a customer-facing AI agent is not purely a research artifact — it is production infrastructure. The following NFR analysis addresses how the architecture meets enterprise-grade requirements, and where explicit design decisions were made to de-risk operational concerns.

5.1 Security

Threat Model Overview

Input Validation & Prompt Injection

The system is queried by AI agents whose inputs originate from untrusted user text. The QueryAnalyzer applies strict pattern matching before any retrieval — EPD and SKU patterns are validated against compiled regexes, preventing injection of malformed identifiers that could cause unexpected behaviour. Query strings are not executed as code at any stage; the retrieval pipeline is a pure data-lookup operation with no `eval()` or dynamic SQL.

5.2 Reliability & Fault Tolerance

The retrieval pipeline has two stateful dependencies: the BM25 index (in memory) and the vector store. Both must have explicit failure modes.

Graceful Degradation

If the dense embedding service is unavailable (GPU OOM, model reload, network partition), the system falls back to BM25-only mode transparently. BM25 alone achieves $MRR=0.881$ — acceptable for continued operation while the dense service recovers. This degradation path is explicitly tested in the evaluation ablation (BM25-only row in the comparison table). The QueryAnalyzer routing layer ensures exact ID lookups continue working regardless of embedding service health.

5.3 Scalability

Current Scale (200 products)

Brute-force numpy matrix multiply: 200×384 float32, $\sim 0.1ms$ per query. BM25Okapi with 641-token vocabulary, $\sim 0.5ms$ per query. Total latency dominated by Python overhead, not index operations. Single-process, in-memory — no distributed infrastructure needed.

Growth Path to Production Scale

Scale	Architecture	Notes
200 products	numpy brute-force + BM25Okapi	Current prototype
10K products	FAISS IVF + BM25Okapi / Elasticsearch	Direct swap, $<1ms$ dense search
100K products	Qdrant HNSW (native hybrid) + sharding	Horizontal shard by category
Multi-tenant	Qdrant namespaces + PostgreSQL metadata	Per-tenant namespace isolation

5.4 Latency

Latency is a first-class concern for AI agent use cases — a retrieval tool call that takes >500ms degrades the perceived responsiveness of the entire agent interaction. The architecture is designed with explicit latency budgets at each stage.

Operation	Latency Budget	Notes
QueryAnalyzer (routing)	<1ms	CPU, pure regex + dictionary lookup
BM25 retrieval (200 products)	<2ms	In-memory index, numpy ops
Dense retrieval (200 products)	<5ms	Matrix multiply on CPU; <1ms on GPU
Qdrant HNSW (100K products)	5–15ms	Approximate NN; tune ef_search vs recall
Cross-encoder re-rank (top-20)	30–80ms	Optional; CPU-bound; GPU cuts to <10ms
Redis cache hit	<1ms	Bypasses retrieval entirely
Total P50 (no cache)	15–50ms	Target for agent tool calls
Total P99 (no cache)	<150ms	Including constraint re-rank

The routing intelligence in QueryAnalyzer is a latency optimisation as much as a quality decision: routing a bare category query to BM25-only avoids an unnecessary embedding inference call (~30–50ms API roundtrip in a production environment), saving budget for the AI agent's downstream LLM call.

5.5 Accuracy & Quality

Accuracy in a retrieval system is multi-dimensional. The evaluation framework addresses each dimension explicitly:

Quality Dimension	What It Measures	Mechanism	Result
Lexical precision	Exact ID and spec value retrieval	BM25 + constraint re-ranker	1.000 on exact queries
Semantic recall	Synonyms, paraphrases, intent matching	Dense bge-small retrieval	1.000 Hit@3 on semantic queries
Constraint adherence	Multi-attribute conjunctive filtering	Post-retrieval re-ranker	1.000 on spec_filtered
Graceful degradation	Unsatisfiable spec combinations	Rules-satisfied count badge	Partial match returned, not empty
Typo robustness	Misspelled queries (cment, cmt)	Synonym expansion + spell correction	Corrected to canonical form

6. Model Evaluation Strategy

Evaluation strategy is where many RAG/retrieval prototypes fail. This system uses a layered evaluation approach that shows the case of our overall system performance.

6.1 Current Evaluation: Synthetic Ground Truth

Challenge: No labelled relevance data is available for this dataset.

Approach: Synthetic ground truth generated deterministically from parsed product fields. 88 evaluated queries (ground truth file also contains 18 human-labelled queries) covering 6 query types with explicit relevance definitions:

Query Type	N Queries	Relevance Definition
Exact EPD lookup	20	Product with that exact EPD ID
Exact SKU lookup	20	Product with that exact SKU
Category vague	10	All 20 products in that category
Manufacturer + category	20	All products from mfr×category intersection
Spec-filtered	13	All products matching the specific parsed value
Multi-attribute	5	Intersection of multiple attribute constraints

6.2 Metrics Selection & Justification

Hit@K: Fraction of queries where at least one relevant product appears in the top-K results. Intuitive for user-facing systems — maps directly to 'did the user find what they needed?'

MRR (Mean Reciprocal Rank): Mean of $1/\text{rank}$ of the first relevant result. Rewards ranking the correct answer higher. $\text{MRR}=1.0$ means the correct answer is always #1; $\text{MRR}=0.5$ means on average the correct answer is at position 2. This is the primary metric for agent use cases where the top result is used as grounding context — position matters.

6.3 Evaluation Roadmap: From Prototype to Production

Phase	Data Source	What It Validates
Phase 1 (Current)	Synthetic GT, 88 queries	Validates mechanics; self-referential for spec queries
Phase 2 (Pre-launch)	Domain expert annotation, ~200 queries	Validates semantic quality; independent ground truth
Phase 3 (Post-launch)	Click-through logs, implicit feedback	Ground truth from real user behaviour
Phase 4 (Ongoing)	A/B testing, shadow deployment	Compare model versions on live traffic before promoting
Phase 5 (Quarterly)	Adversarial queries, edge case sweep	Red-team the system; find failure modes proactively

7. Retrieval Improvement Roadmap

The current system achieves near-perfect metrics on synthetic evaluation. The improvement roadmap addresses the gap between synthetic and real-world performance, and the path to production-grade quality across a growing, dynamic catalogue.

7.1 Immediate Improvements (Within Current Architecture)

Improvement	Effort	Expected Gain	Notes
Query expansion (domain synonyms)	Low	+5–10% recall	Map 'rebar' → 'B500B reinforcing bar'; 'ETICS' → 'insulation render system'
Cross-encoder re-ranker	Medium	+5–15% MRR	ms-marco-MiniLM or ColBERT re-ranks top-20; highest gain on semantic queries
Larger embedding model (bge-base)	Low	+3–8% MRR	Swap model constant; requires re-embedding; ~110M params vs 33M
Spelling correction	Low	+2–5% on typo queries	pyspellchecker with domain vocab; already partially implemented
RRF k-constant tuning	Medium	+2–5% overall	Optimise k on a held-out validation set; currently hardcoded at 60

7.2 Data-Driven Improvements

The highest-value retrieval improvement is not a better algorithm — it is better training signal. Even 500 labelled (query, product_id) pairs from real users would enable embedding fine-tuning that could improve MRR by 15–25%.

- **Option 1 — Synthetic labels:** LLM-generated synthetic queries: Prompt Claude/GPT to generate 10 diverse queries per product (vague to specific). Creates ~2,000 labelled training pairs.
- **Option 2 — Expert annotation:** Domain expert annotation: 2–3 hours with a building materials procurement specialist. Produces high-quality labels with appropriate industry jargon coverage.
- **Option 3 — Implicit feedback:** Click-through logging: Deploy current system, log queries with user interactions (clicked product, skipped results, search refinements). Accumulate labels passively.
- **Option 4 — Active learning:** Active learning: Log queries with low re-ranker confidence scores; priorities those for human annotation. Maximize label quality per annotation hour.

7.3 Architectural Improvements for Production

Improvement	Mechanism	Trade-off / Consideration
Structured metadata pre-filter	Qdrant payload filter	Eliminates cross-category false positives at query time; not post-retrieval
Knowledge graph integration	Manufacturer → product family → substitutes	Enables substitution queries; requires knowledge engineering investment
Personalised re-ranking	User purchase history as ranking signal	Procurement context awareness; requires user identity and history store
LLM query rewriting	GPT-4o rewrites ambiguous queries before retrieval	+5–15% on vague queries; adds 300–500ms latency

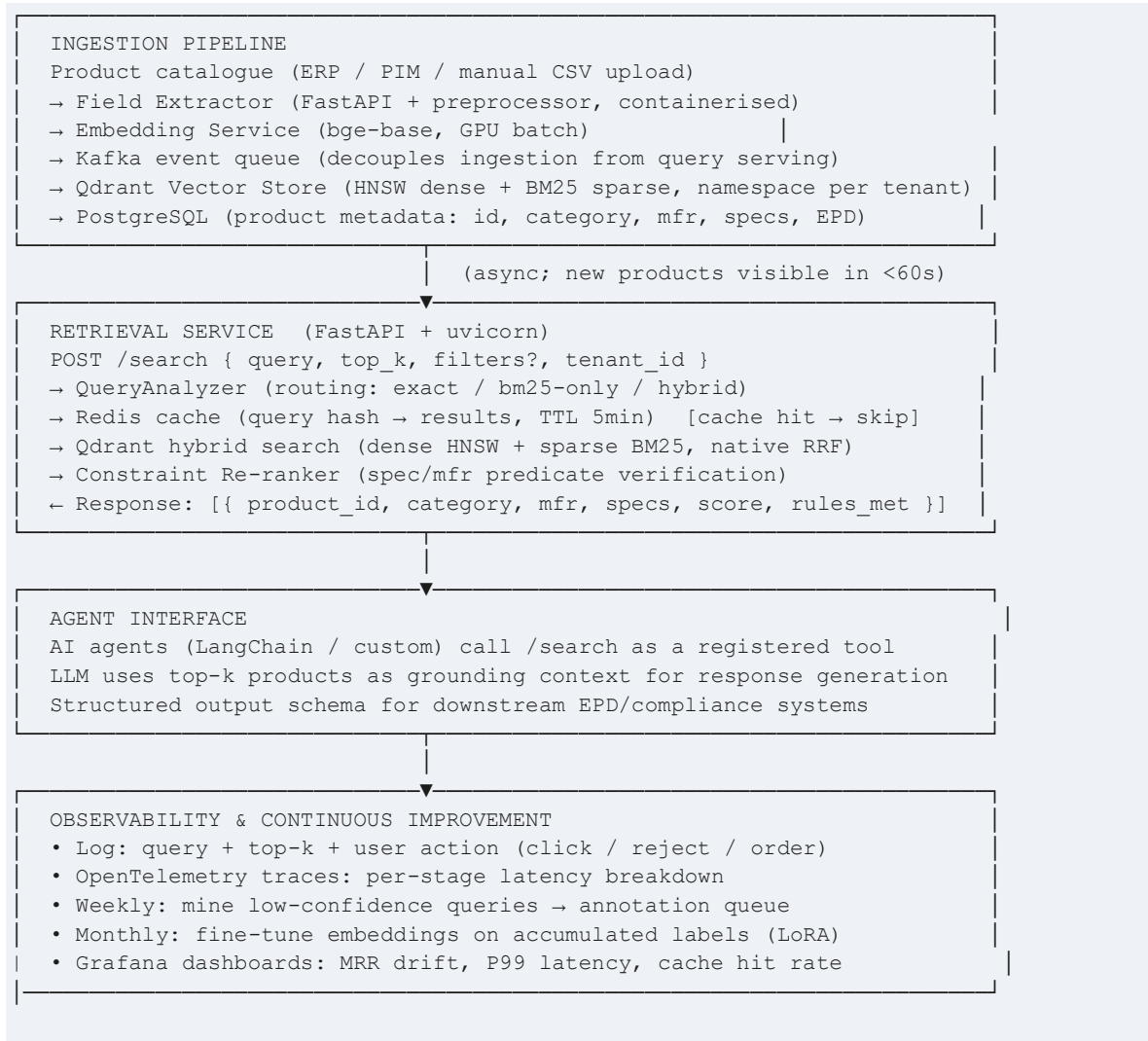
7.4 Open-Source vs. Hosted Embedding Providers

Dimension	Open-Source (bge/MiniLM)	Hosted API (OpenAI/Cohere)
Cost at scale	Compute only (GPU amortised)	Per-token pricing (e.g. \$0.02/1M tokens)
Latency	<1ms on GPU; ~5ms on CPU	50–200ms API roundtrip (network + queue)
Data privacy	Data stays on-premises	Product data sent to third-party servers
Fine-tuning control	Full — PEFT/LoRA, custom loss	Provider-dependent (OpenAI: no; Cohere: managed API)
Maintenance burden	Model versioning, re-deployment	Provider-managed; zero ops
Quality ceiling	Open SOTA: bge-large, E5-mistral	Access to larger proprietary models (ada-003)
Recommended for	Regulated industries, high volume	Prototyping, low-volume, no GPU budget

8. Proposed Production SaaS Architecture

The following architecture is designed for multi-tenant SaaS deployment serving AI agents across multiple client organizations, each with their own product catalogues.

8.1 System Diagram



8.2 Technology Stack Justifications

Component	Choice	Justification
Vector DB	Qdrant	Native hybrid search (dense HNSW + BM25 sparse, built-in RRF); self-hostable; payload filtering; Docker-first deployment; strong Python client.
Sparse Index	Qdrant BM25 / Elasticsearch	Mature, scalable. Elasticsearch preferred if full-text search beyond retrieval is needed (filtering, aggregations).
Cache Layer	Redis Sentinel	Sub-millisecond lookup; most procurement queries repeat (same

		SKU, same EPD); TTL-based invalidation on catalogue updates.
Embedding Serving	bge-base	Batched GPU inference; dynamic batching across replicas; model versioning; supports TensorRT optimisation.
API Layer	FastAPI + uvicorn	Async ASGI; auto-generated OpenAPI spec for agent tool definitions; native Pydantic validation; supports WebSockets for streaming.
Message Queue	Apache Kafka	Durable event log for catalogue updates; enables multiple downstream consumers (embedding job, metadata DB, audit log).
Metadata Store	PostgreSQL (RDS Multi-AZ)	Structured product metadata for non-vector queries; JSONB for flexible spec storage; foreign keys enforce catalogue integrity.
Orchestration	Kubernetes (EKS/GKE)	Horizontal pod autoscaling; rolling deployments; namespace isolation per environment.
Model Registry	MLflow or HF Hub (private)	Versioned model artefacts; experiment tracking for fine-tuning runs; SHA-pinned deployment.
Observability	OpenTelemetry + Grafana + Langsmith	End-to-end trace per query; per-stage latency; MRR dashboards; anomaly alerting.
Secret Management	HashiCorp Vault / AWS Secrets Manager	API keys, database credentials, model access tokens — never in environment variables or code.

9. Technology Decisions Summary

Decision Area	Chosen	Rejected Alternative	Key Reason
Preprocessing	Per-format parsers + universal regex	Embed raw text directly	Clean input → significantly better embedding quality
Sparse retrieval	BM25Okapi (k1=1.5, b=0.75)	TF-IDF	TF saturation + length normalisation for variable-length docs
Dense model	bge-small-en-v1.5	all-MiniLM-L6-v2, OpenAI ada	Top BEIR score at small size; asymmetric retrieval; free; local
Vector storage	numpy float32 matrix	FAISS, Chroma, Pinecone	200 products; brute-force is exact, <1ms, zero dependency
Fusion	RRF (k=60)	Weighted sum $\alpha \times \text{BM25} + (1 - \alpha) \times \text{dense}$	Scale-invariant; no tuning parameter; industry standard
Query routing	Rule cascade	ML classifier	No training data; deterministic; debuggable; zero false positives on IDs
Re-ranking	Constraint predicate verifier	Cross-encoder (production path)	Exact predicate satisfaction vs. BM25 term frequency approximation
Evaluation	Synthetic GT + human-labelled subset	Manual annotation only	Zero cost; deterministic; honest about limitations
Demo	Jupyter	Jupyter widgets	Fastest to build; polished; runs locally

Appendix A — Notebook Execution Screenshots

The following screenshots show the executed Jupyter notebook: lca_product_retrieval.ipynb. Each section corresponds to a major pipeline stage, from data loading and EDA through preprocessing, retrieval engine construction, example searches, and final evaluation.

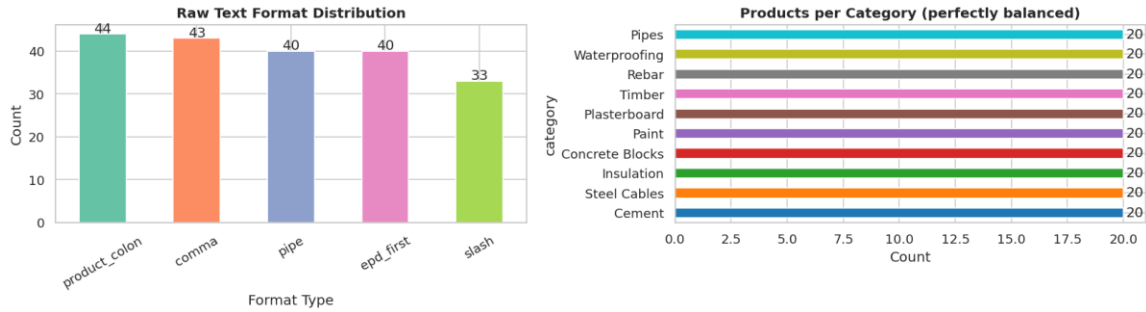


Figure A.1 — EDA Overview: dataset statistics, category distribution, format detection across 200 products

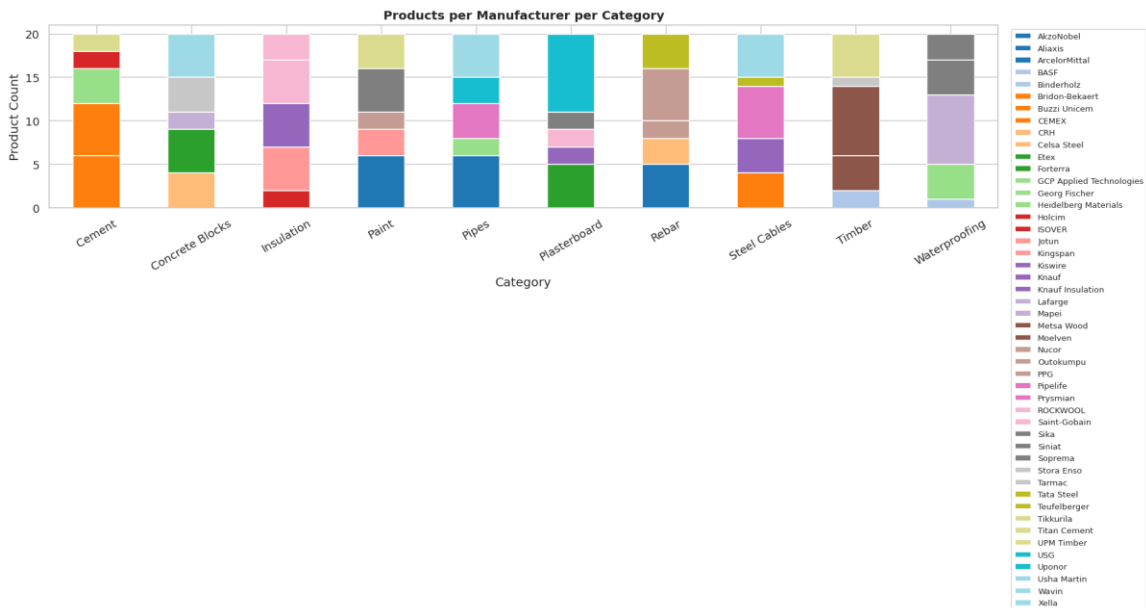


Figure A.2 — Manufacturer Distribution: 48 unique manufacturers across 10 categories

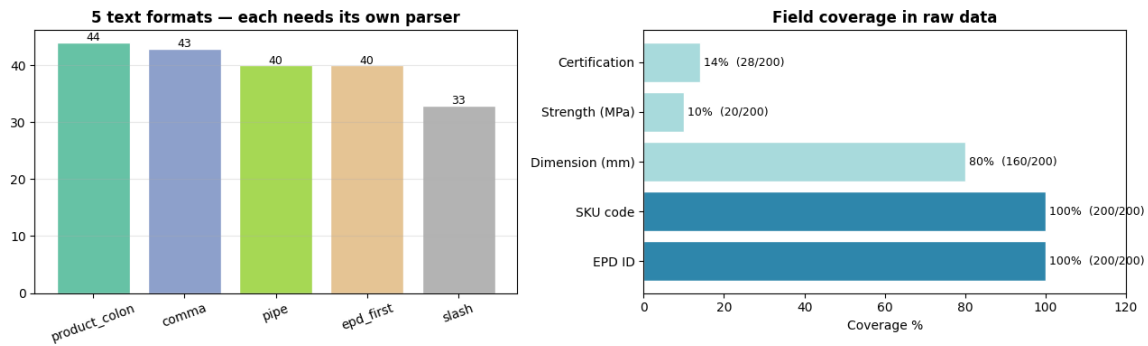


Figure A.3 — Notebook Cell 4 Output: raw data loading confirmation and initial dataset statistics (200 products, 2 columns)

```
# Each tuple is (query_string, description_for_reader)
examples = [
    ("EPD202400001", "Exact EPD lookup"),
    ("CEM-1003", "Exact SKU lookup"),
    ("cement", "Category browse"),
    ("cement Heidelberg Materials", "Category + manufacturer filter"),
    ("steel cables 12mm 1770 MPa", "Multi-spec filter"),
    ("insulation A1 fire class 120mm", "Spec + certification filter"),
    ("waterproof membrane for basement walls", "Semantic / natural language"),
]

for query, label in examples:
    print(f"\n>>> {label}")
    display(search(query))
```

Figure A.4 — Testing sample on multiple use-case

```
>>> Exact EPD lookup
```

```
Query   : EPD202400001
Intent  : type=exact_epd | routing=exact
```

#	ID	Category	Manufacturer	EPD	Specs
1	P0001	Cement	Heidelberg Materials	EPD202400001	25.0kg C30/37

```
>>> Exact SKU lookup
```

```
Query   : CEM-1003
Intent  : type=exact_sku | routing=exact
```

#	ID	Category	Manufacturer	EPD	Specs
1	P0003	Cement	Holcim	EPD202400003	25.0kg CEM I 42.5N

Figure A.4.1 — Output1

```
>>> Category browse
```

```
Query   : cement
Intent  : type=category_vague | routing=bm25_only | category=Cement
```

#	ID	Category	Manufacturer	EPD	Specs
1	P0007	Cement	Titan Cement	EPD202400007	40.0kg CEM I 42.5N
2	P0019	Cement	Titan Cement	EPD202400019	25.0kg CEM I 42.5N
3	P0002	Cement	Heidelberg Materials	EPD202400002	25.0kg C25/30
4	P0001	Cement	Heidelberg Materials	EPD202400001	25.0kg C30/37
5	P0016	Cement	Buzzi Unicem	EPD202400016	25.0kg C30/37

```
>>> Category + manufacturer filter
```

```
Query   : cement Heidelberg Materials
Intent  : type=manufacturer_filtered | routing=hybrid | category=Cement | mfr=Heidelberg Materials | rules=
```

#	ID	Category	Manufacturer	EPD	Specs
1	P0001	Cement	Heidelberg Materials	EPD202400001	25.0kg C30/37 [1/1 rules]
2	P0013	Cement	Heidelberg Materials	EPD202400013	25.0kg C30/37 [1/1 rules]
3	P0002	Cement	Heidelberg Materials	EPD202400002	25.0kg C25/30 [1/1 rules]
4	P0011	Cement	Heidelberg Materials	EPD202400011	25.0kg CEM II/A-L 32.5R [1/1 rules]
5	P0020	Cement	CEMEX	EPD202400020	40.0kg C30/37 [0/1 rules]

Figure A.4.2 — Output2

```
>>> Multi-spec filter
```

```
Query   : steel cables 12mm 1770 MPa
Intent  : type=spec_filtered | routing=hybrid | category=Steel Cables | specs=['12mm', '1770 MPa'] | rules=
```

#	ID	Category	Manufacturer	EPD	Specs
1	P0036	Steel Cables	Teufelberger	EPD202400036	12.0mm 1770.0MPa [2/2 rules]
2	P0024	Steel Cables	Kiswire	EPD202400024	6.0mm 1770.0MPa [1/2 rules]
3	P0029	Steel Cables	Usha Martin	EPD202400029	8.0mm 1770.0MPa [1/2 rules]
4	P0030	Steel Cables	Bridon-Bekaert	EPD202400030	10.0mm 1770.0MPa [1/2 rules]
5	P0034	Steel Cables	Prysmian	EPD202400034	10.0mm 1770.0MPa [1/2 rules]

```
>>> Spec + certification filter
```

```
Query   : insulation A1 fire class 120mm
Intent  : type=spec_filtered | routing=hybrid | category=Insulation | specs=['120mm'] | rules=['120mm', 'A1']
```

#	ID	Category	Manufacturer	EPD	Specs
1	P0041	Insulation	Knauf Insulation	EPD202400041	120.0mm [2/2 rules]
2	P0054	Insulation	ROCKWOOL	EPD202400054	120.0mm [2/2 rules]
3	P0047	Insulation	Knauf Insulation	EPD202400047	80.0mm [1/2 rules]
4	P0049	Insulation	Saint-Gobain	EPD202400049	50.0mm [1/2 rules]
5	P0051	Insulation	ROCKWOOL	EPD202400051	80.0mm [1/2 rules]

Figure A.4.3 — Output3

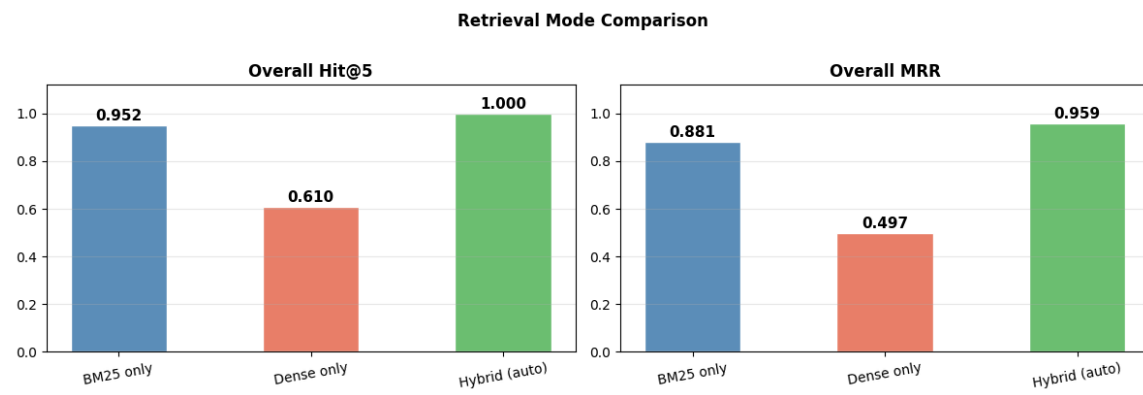


Figure A.6 — Notebook Cell 22 Output: Evaluation results — Hit@K and MRR comparison across BM25-only, Dense-only, and Hybrid retrieval modes